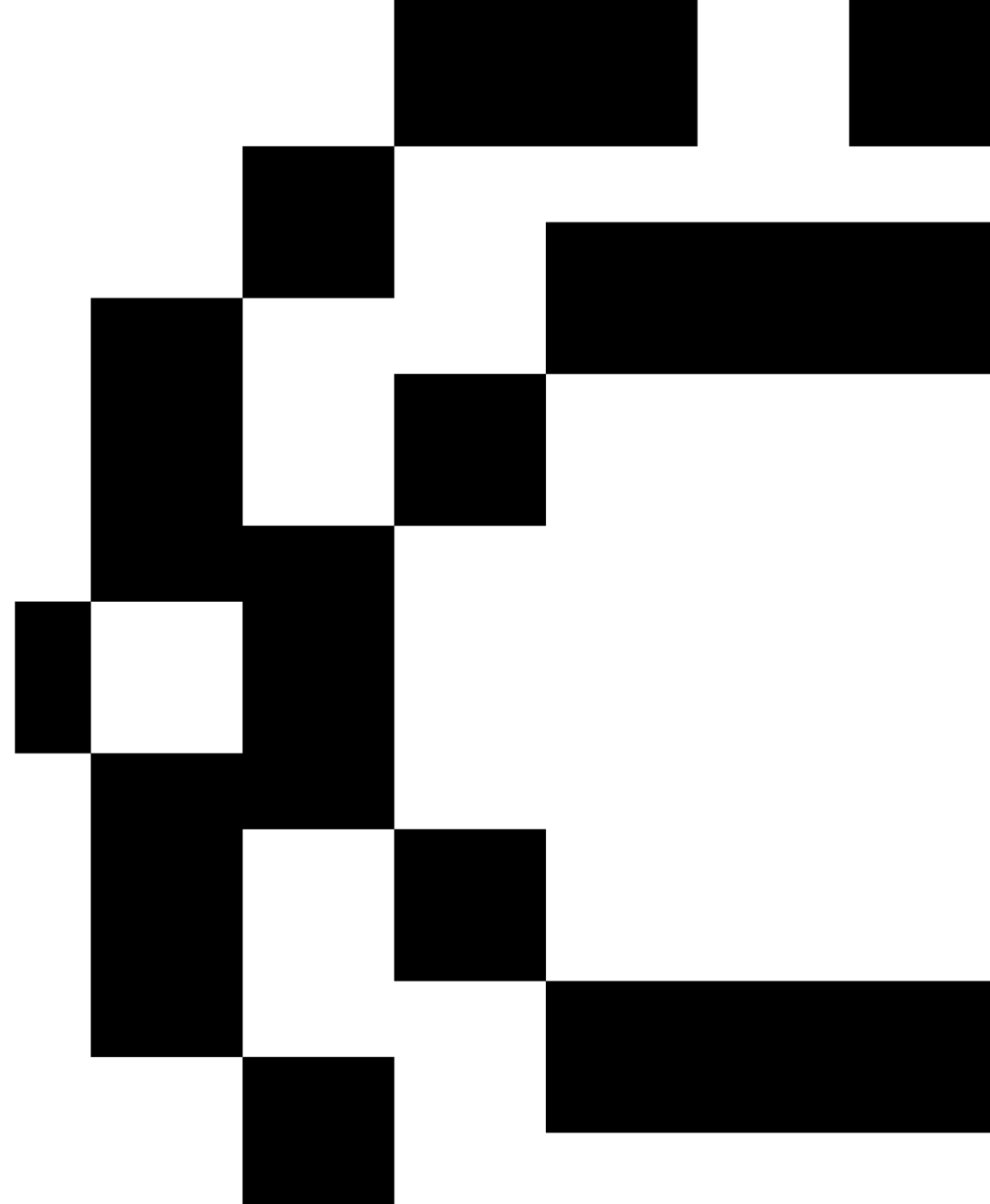


Word Representations

Text Mining 2025-2026

Bruno Jardim

bjardim@novaims.unl.pt



Lecture Plan

- 01. Classification with BoW
- 02. Word Representations
- 03. Word Embeddings (Word2Vec)
- 04. Classification with Word2Vec



01. Classification with BoW

Recall Problems with Bag of Words

Classification with BoW

Recall BoW

Review 1 – “Great movie”

Review 2 – “Bad movie”

	great	movie	bad
R1	1	1	0
R2	0	1	1

Task:

Predict the sentiment of movie reviews using a neural network with sigmoid activation function.

Recall Problems with Bag of Words

Classification with BoW

Recall BoW

Review 1 – “Great movie”

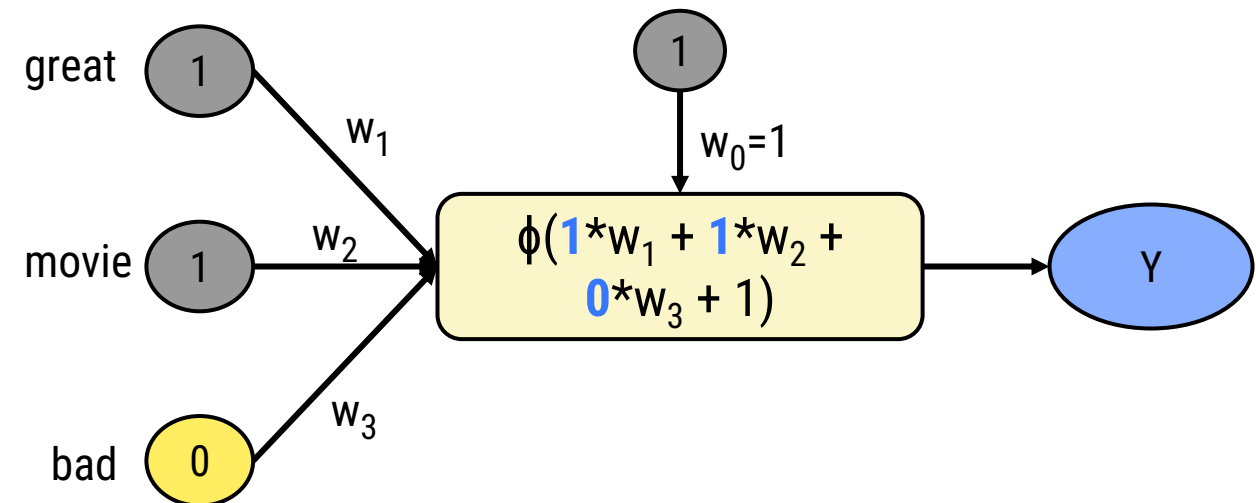
Review 2 – “Bad movie”

	great	movie	bad
R1	1	1	0
R2	0	1	1

Task:

Predict the sentiment of movie reviews using a neural network with sigmoid activation function.

Review 1:



Recall Problems with Bag of Words

Classification with BoW

Recall BoW

Review 1 – “Great movie”

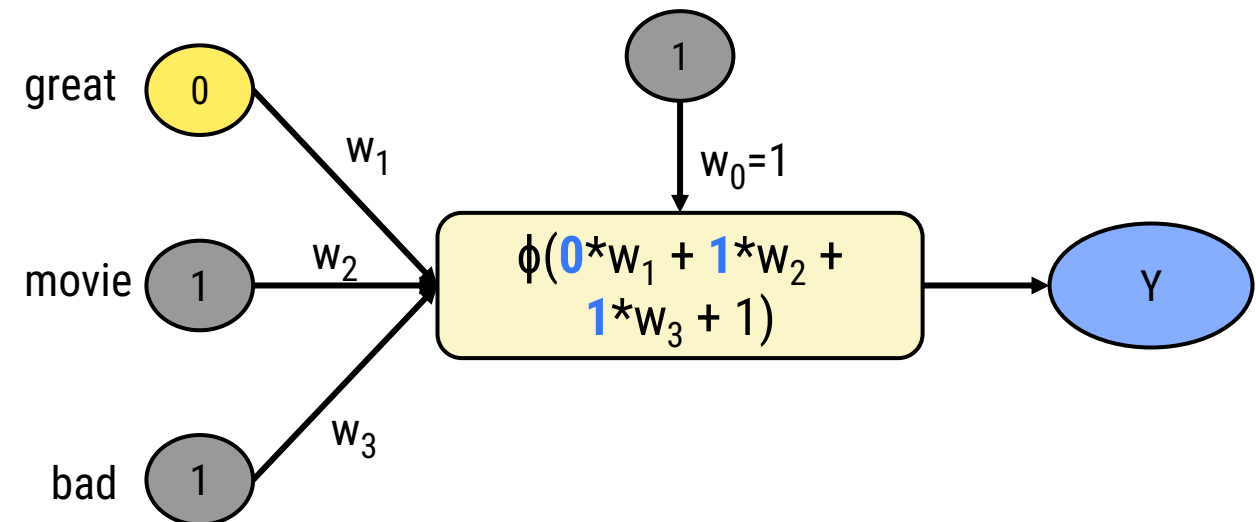
Review 2 – “Bad movie”

	great	movie	bad
R1	1	1	0
R2	0	1	1

Task:

Predict the sentiment of movie reviews using a neural network with sigmoid activation function.

Review 2:





02.

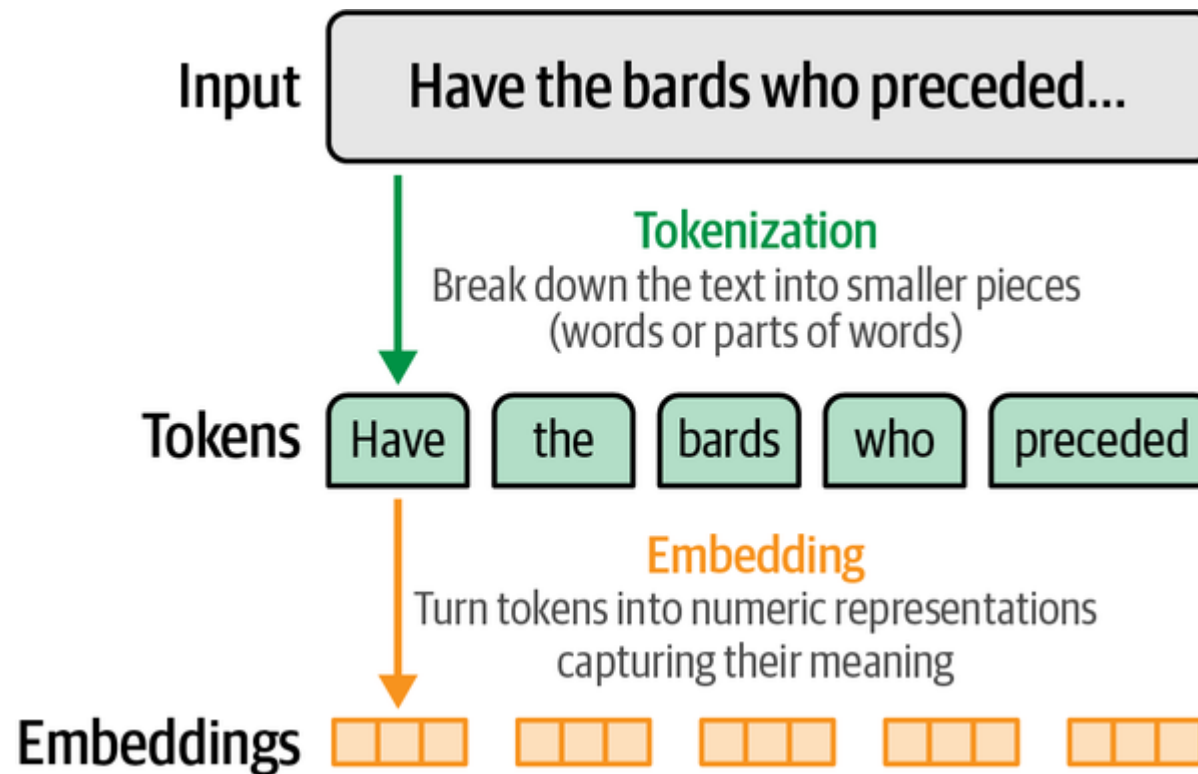
Word Representations

1. Curse of dimensionality
2. No semantic relationships
(Word order is discarded)
3. All words have the same importance
4. Context changes meaning

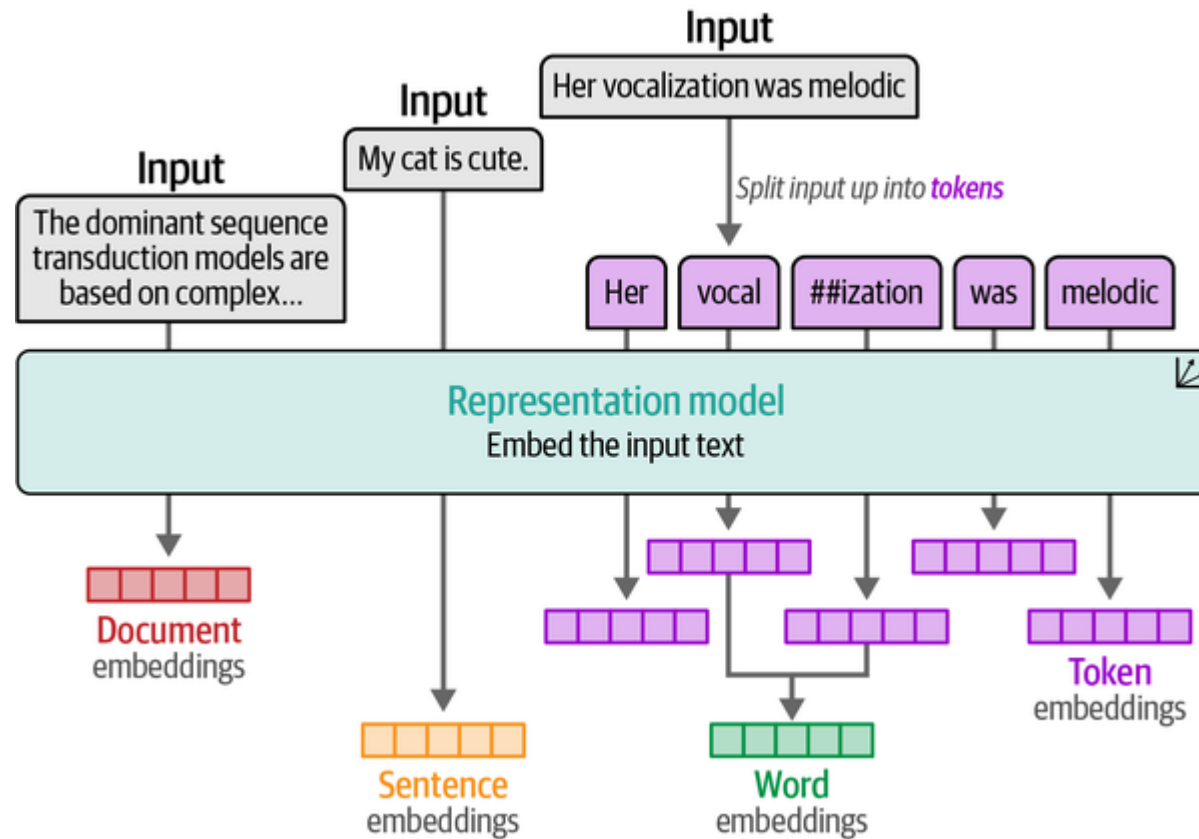
1. Curse of dimensionality
2. No semantic relation
(Word order)
3. All words are same importance
4. Context changes meaning

But how to solve it?

Use word/token representations instead of document ones!



Use word/token representations instead of document ones!



Recall Problems with Bag of Words

Word Representations

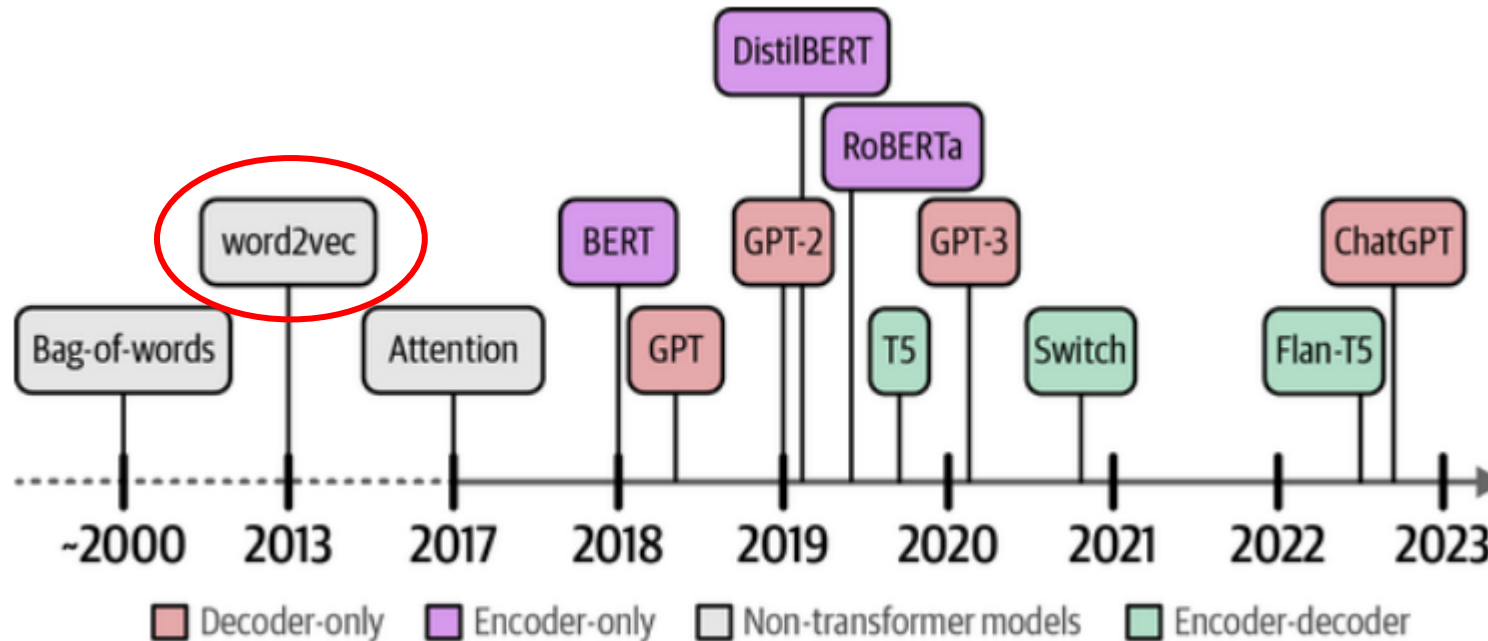


Figure 1-1. A peek into the history of Language AI.

Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

Word representations capture semantic relationships

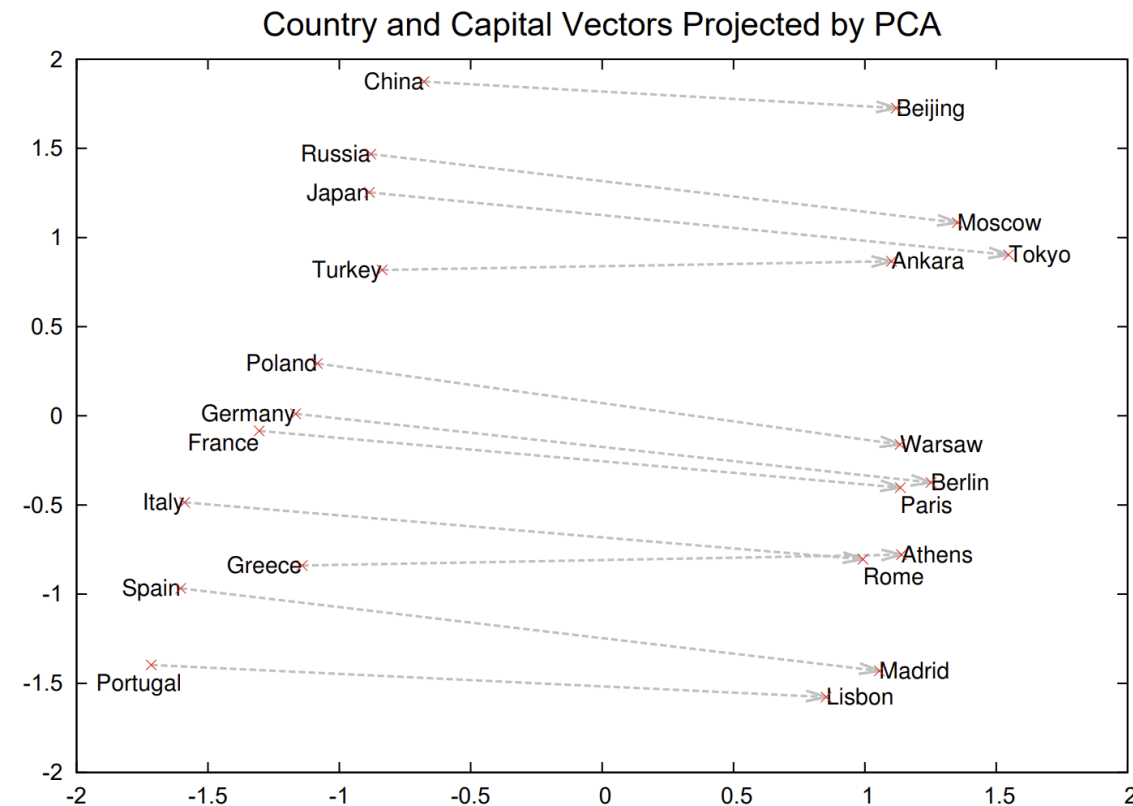
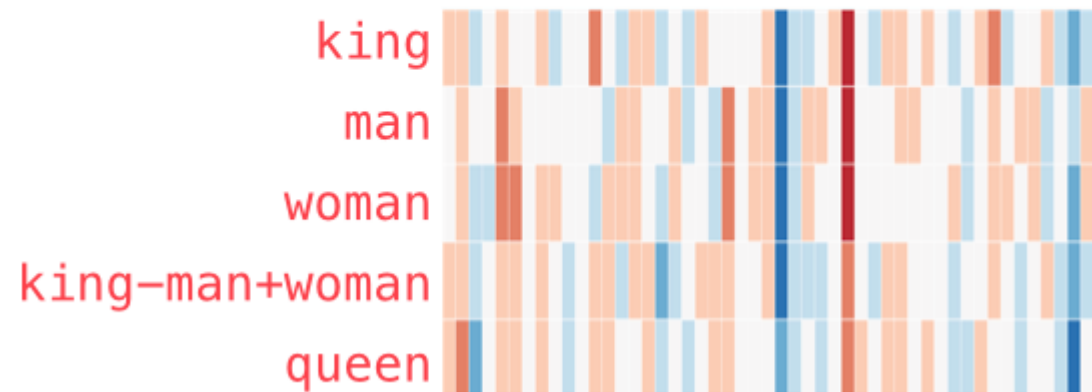


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

Word representations capture semantic relationships

The resulting vector from "king-man+woman" almost exactly equals "queen".

king - man + woman $\approx$ queen



Some ways to generate word vectors:

- Term-Term matrix
- Point-wise Mutual Information (PMI)
- Skip-gram (word2vec)

Some ways to generate word vectors:

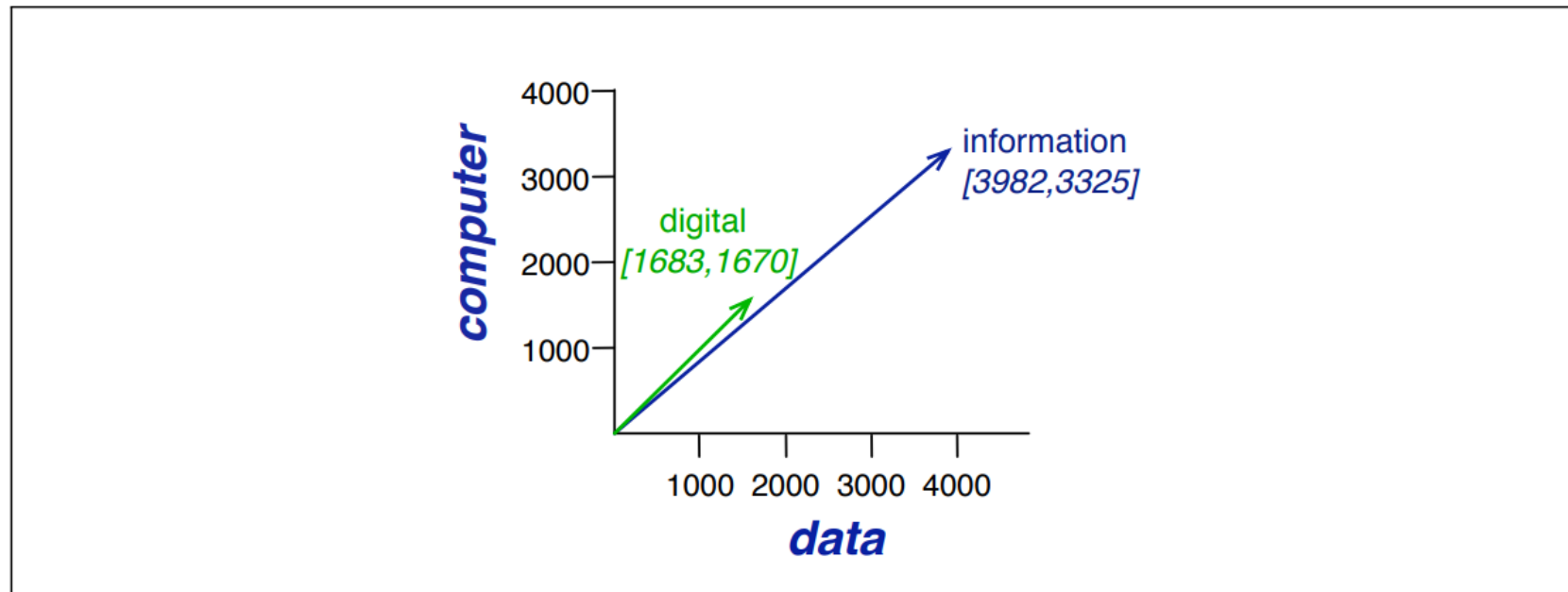
- **Term-term matrix** is of dimensionality $|V| \times |V|$ and each cell records the number of times the row word (target) and the column word (context) co-occur in the training corpus

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Some ways to generate word vectors:

- A spatial visualization of word vectors for digital and information, showing just two of the dimensions, corresponding to the words data and computer.



Some ways to generate word vectors:

- **Point-wise Mutual Information (PMI)** is a measure of how often two events x and y occur, compared with what we would expect if they were independent:

$$I(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)}$$

- The pointwise mutual information between a target word w and a context word c is then defined as:

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- It is common to use Positive PMI (called PPMI) which replaces all negative PMI values with zero:

$$\text{PPMI}(w, c) = \max\left(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0\right)$$

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI)
- word = **information**, context = **data**:

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI) between **information** and **data**:

1- Calculate co-occurrence matrix

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI) between **information** and **data**:

2- Calculate word probability

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$P(w=\text{information}) = \frac{7703}{11716} = .6575$$

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI) between **information** and **data**:

2- Calculate context probability

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$P(w=\text{information}) = \frac{7703}{11716} = .6575$$

$$P(c=\text{data}) = \frac{5673}{11716} = .4842$$

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI):

3- Calculate co-occurrence probability

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$P(w=\text{information}, c=\text{data}) = \frac{3982}{11716} = .3399$$

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI):

4- Create Probabilities Matrix

$$P(w=\text{information}, c=\text{data}) = \frac{3982}{11716} = .3399$$

$$P(w=\text{information}) = \frac{7703}{11716} = .6575$$

$$P(c=\text{data}) = \frac{5673}{11716} = .4842$$

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI):

5- Calculate PPMI:

$$\text{PPMI}(w, c) = \max\left(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0\right)$$

$$\text{PPMI}(\text{information}, \text{data}) = \log_2(.3399 / (.6575 * .4842)) = .0944$$

Point-wise Mutual Information (PMI)

Some ways to generate word vectors:

- Calculating Positive Point-wise Mutual Information (PPMI):

5- Calculate PPMI:

$$\text{PPMI}(\text{information}, \text{data}) = \log_2(.3399 / (.6575 * .4842)) = .0944$$

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

Only 2 Problems with Bag of Words are solved Word Representations

1. Curse of dimensionality 
2. No semantic relationships
(Word order is discarded) 
3. All words have the same importance 
4. Context changes meaning 



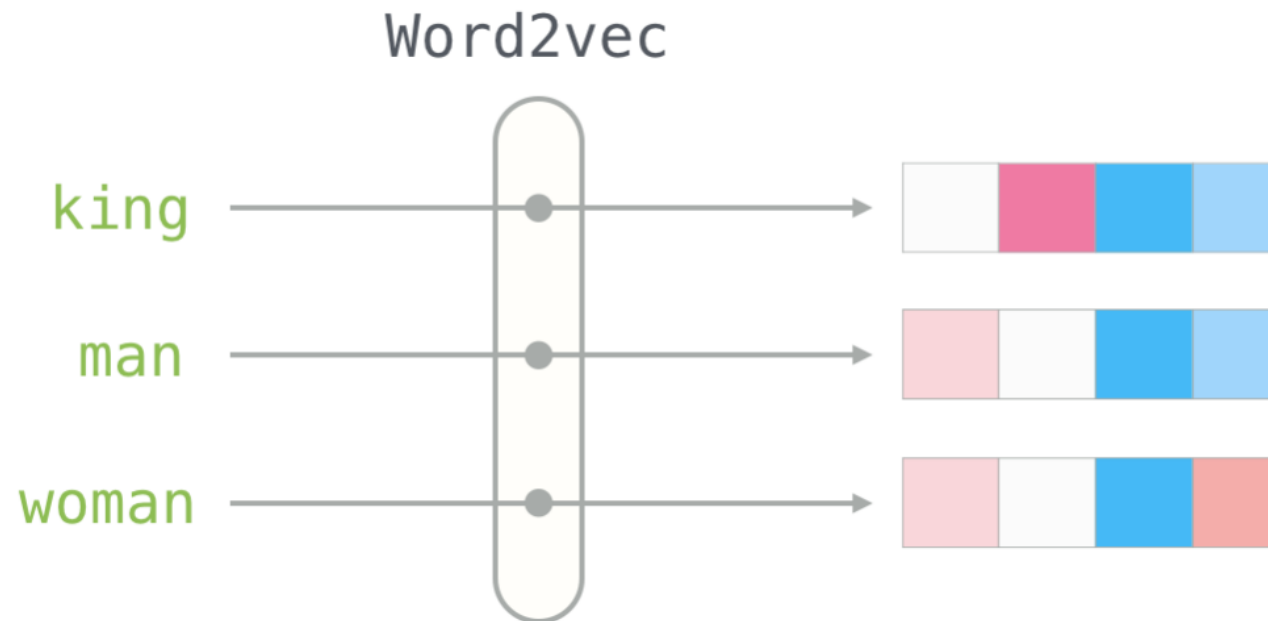
03. Word Embeddings (Word2Vec)

Word2Vec method solves the problem of Dimensionality

Word Embeddings

1. Curse of dimensionality ✓
2. No semantic relationships
(Word order is discarded) ✓
3. All words have the same importance ✓
4. Context changes meaning ✗

Skip-gram (also known as **word2vec**) is a method to estimate **short dense word vectors**.



How it works:

- The intuition is that **similar words tend to appear together**.
- We **train a classifier** (like simple neural network with a single hidden layer) to perform a binary prediction task: “**Is word w likely to show up near c ?**”
- We’re not actually going to use that classifier for the task we trained it on.
- The goal is just to **learn the weights of the hidden layer** – these weights are the word vectors that we’re trying to learn.

1. Define word and context:

For each **word** in a corpus, select the **context window** – considering, for example, a **context of 2 words**:

“(...) Money is a terrible master but an excellent servant (...)”

1. Define word and context:

For each **word** in a corpus, select the **context window** – considering, for example, a **context of 2 words**:

Money is a terrible **master** but an excellent servant

Money is a terrible master **but** an excellent servant

Money is a terrible master but **an** excellent servant

Money is a terrible master but an **excellent** servant
etc.

Word	Context	Target
master	terrible	1
master	but	1
but	master	1
but	an	1
an	but	1
an	excellent	1
excellent	an	1
excellent	servent	1

Etc...

2. Add negative samples

Add **negative samples**: words that never appear in the same context window. The logic behind this, is that both positive and negative observations are required to train a model.

Word	Context	Target
master	terrible	1
master	ring	0
master	but	1
but	master	1
but	late	0
but	an	1
an	crop	0
an	but	1

Etc...

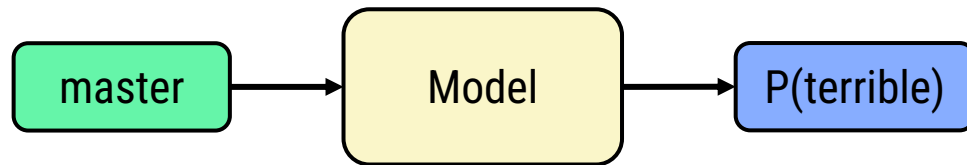
3. Define task

Now the model needs to solve the following task: **given a word w and a context word c , predict if this combination of w and c occurred in the corpus** (i.e. if target is 1 or 0):

Word	Context	Target
master	terrible	?
master	ring	?
master	but	?
but	master	?
but	late	?
but	an	?
an	crop	?
an	but	?

3. Define task

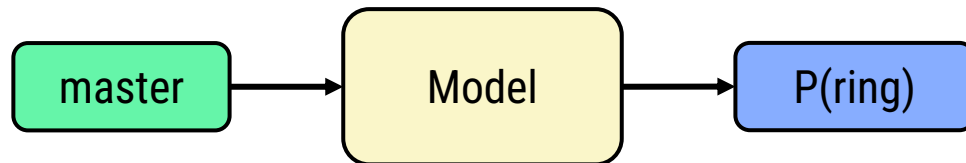
Now the model needs to solve the following task: given a word w and a context word c , predict if this combination of w and c occurred in the corpus (i.e. if target is 1 or 0):



Word	Context	Target
master	terrible	1
master	ring	?
master	but	?
but	master	?
but	late	?
but	an	?
an	crop	?
an	but	?

3. Define task

Now the model needs to solve the following task: given a word w and a context word c , predict if this combination of w and c occurred in the corpus (i.e. if target is 1 or 0):

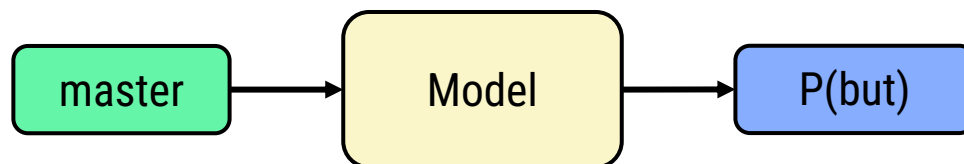


Word	Context	Target
master	terrible	1
master	ring	0
master	but	?
but	master	?
but	late	?
but	an	?
an	crop	?
an	but	?

3. Define task

Now the model needs to solve the following task: **given a word w and a context word c , predict if this combination of w and c occurred in the corpus** (i.e. if target is 1 or 0):

Word	Context	Target
master	terrible	1
master	ring	0
master	but	1
but	master	?
but	late	?
but	an	?
an	crop	?
an	but	?



Etc...

3. Define task

Once the model is trained, its **weights are used to represent the words**

Model
(trained)



Weight	Value
1	0.234
2	1.298
3	2.762
4	0.567
5	1.222
6	0.334
(...)	

4. Train the model

Example training input **P(terrible|the)**

Define input:

Input size

$1 \times v$, where v is the size of the vocabulary

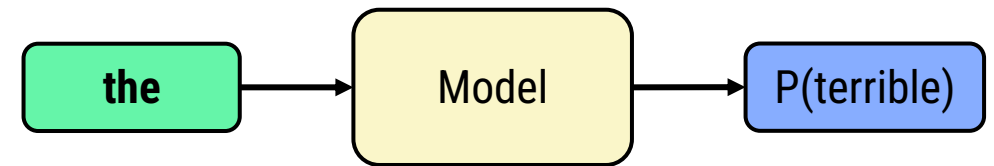
	(...)	the	master	money	(...)
(...)	(...)	0	0	0	(...)
the	(...)	1	0	0	(...)
master	(...)	0	1	0	(...)
money	(...)	0	0	1	(...)
But	(...)	0	0	0	(...)
an	(...)	0	0	0	(...)
0	(...)	(...)	(...)	(...)	(...)



Size of vocabulary (v)

Input vector of "the" (with size of vocabulary):

$[0, 1, 0, 0, 0, (...), X_v]$



4. Train the model

Example training input **P(terrible|master)**

Define input:

Input size

$1 \times v$, where v is the size of the vocabulary

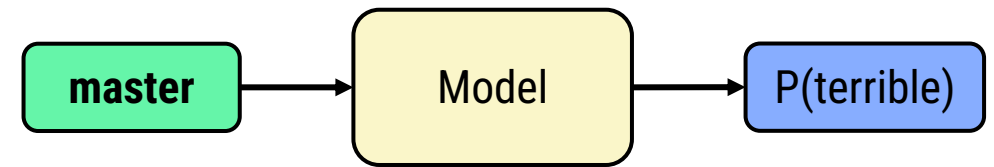
	(...)	the	master	money	(...)
(...)	(...)	0	0	0	(...)
the	(...)	1	0	0	(...)
master	(...)	0	1	0	(...)
money	(...)	0	0	1	(...)
But	(...)	0	0	0	(...)
an	(...)	0	0	0	(...)
(...)	(...)	(...)	(...)	(...)	(...)

Input vector of "master" (with size of vocabulary):

$[0, 0, 1, 0, 0, (...), X_v]$



Size of vocabulary (v)

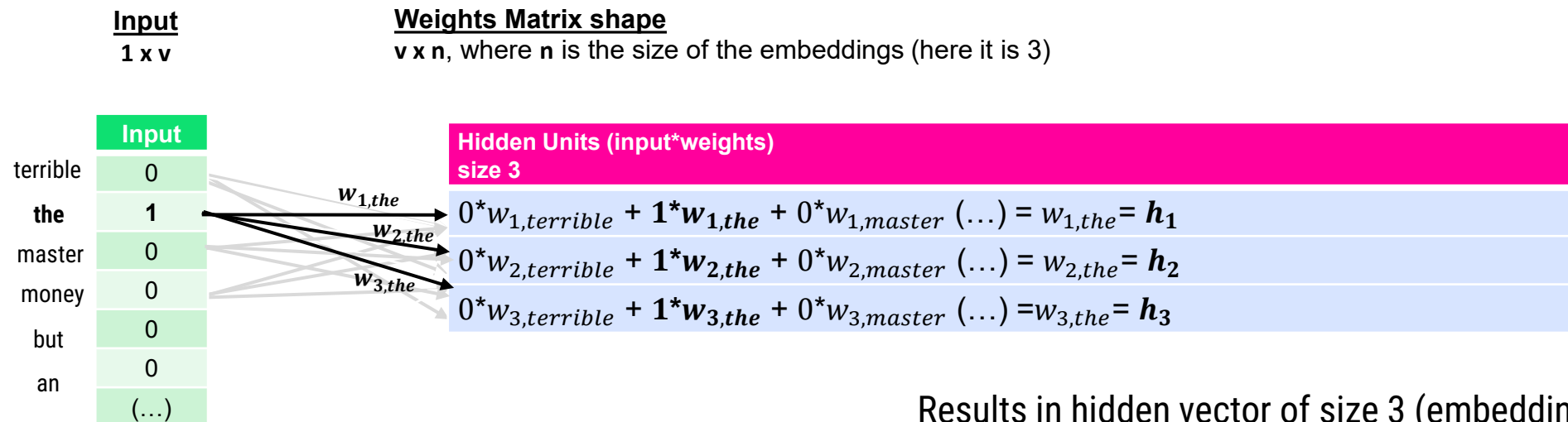
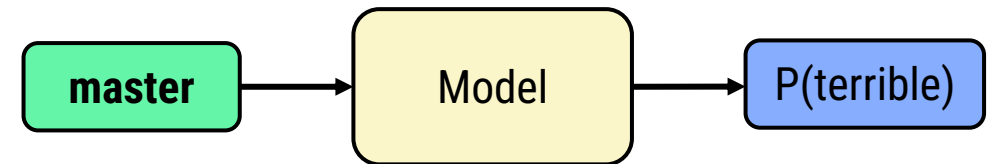


4. Train the model

Define the **size of the weight matrix (W)**, this will be our embedding size, i.e. size of word vector to be retrieved

In the example below **W is size 3**

*(weights' values can be initialized randomly)



Results in hidden vector of size 3 (embedding size):

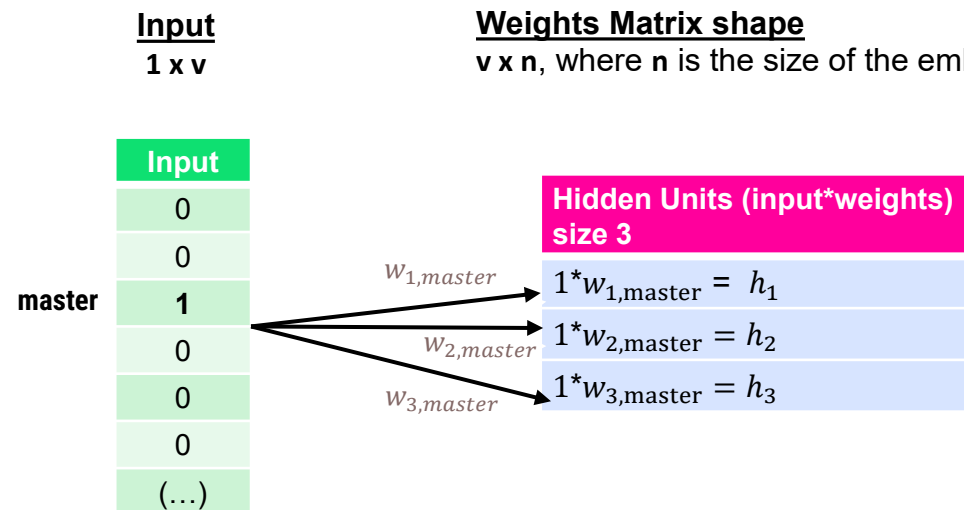
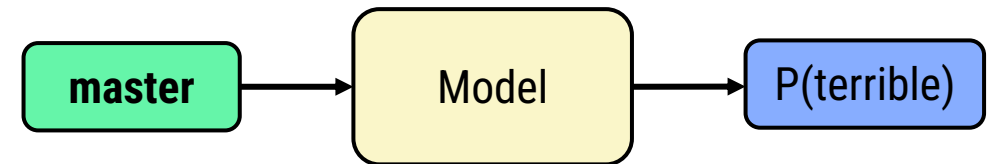
$$[w_{1,the}, w_{2,the}, w_{3,the}] = [h_1, h_2, h_3]$$

4. Train the model

Define the **size of the weight matrix (W)**, this will be our embedding size, i.e. size of word vector to be retrieved

In the example below **W is size 3**

*(weights' values can be initialized randomly)



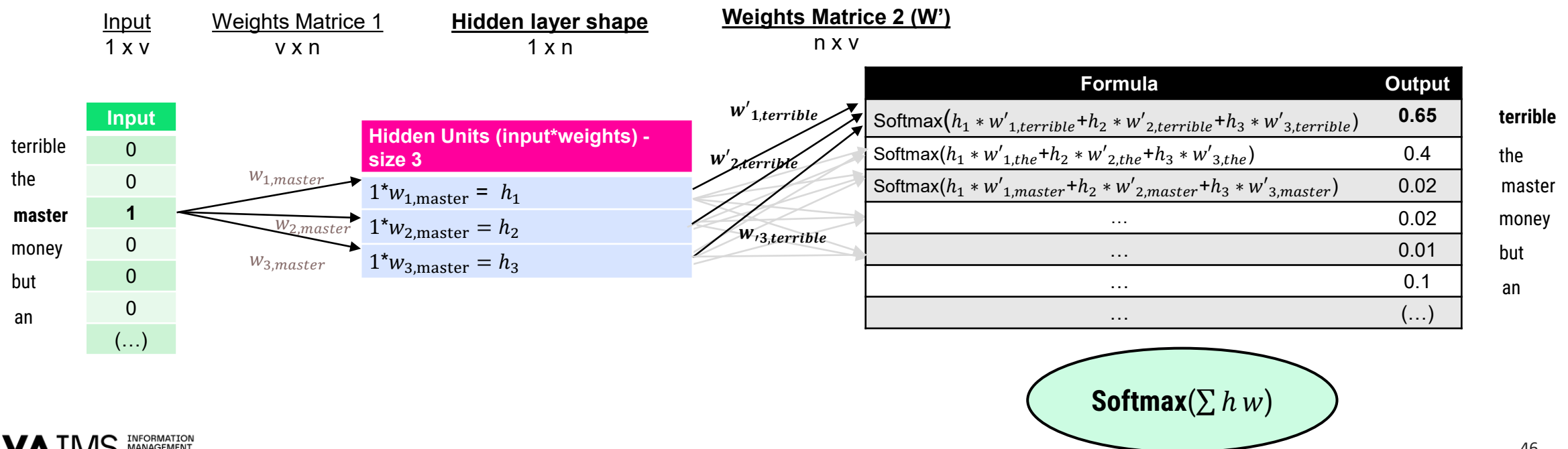
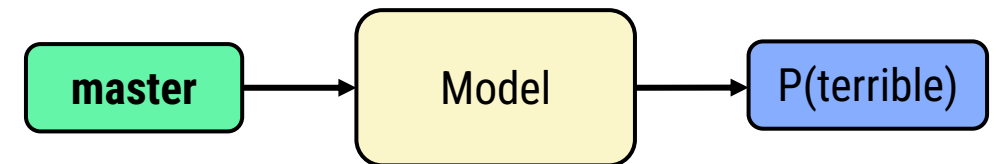
Hidden vector of size 3 (embedding size):

$$[w_{1, master}, w_{2, master}, w_{3, master}] = [h_1, h_2, h_3]$$

4. Train the model

Multiple by a **second matrix of weights**, sum the results and transform it into **probabilities for each possible word**.

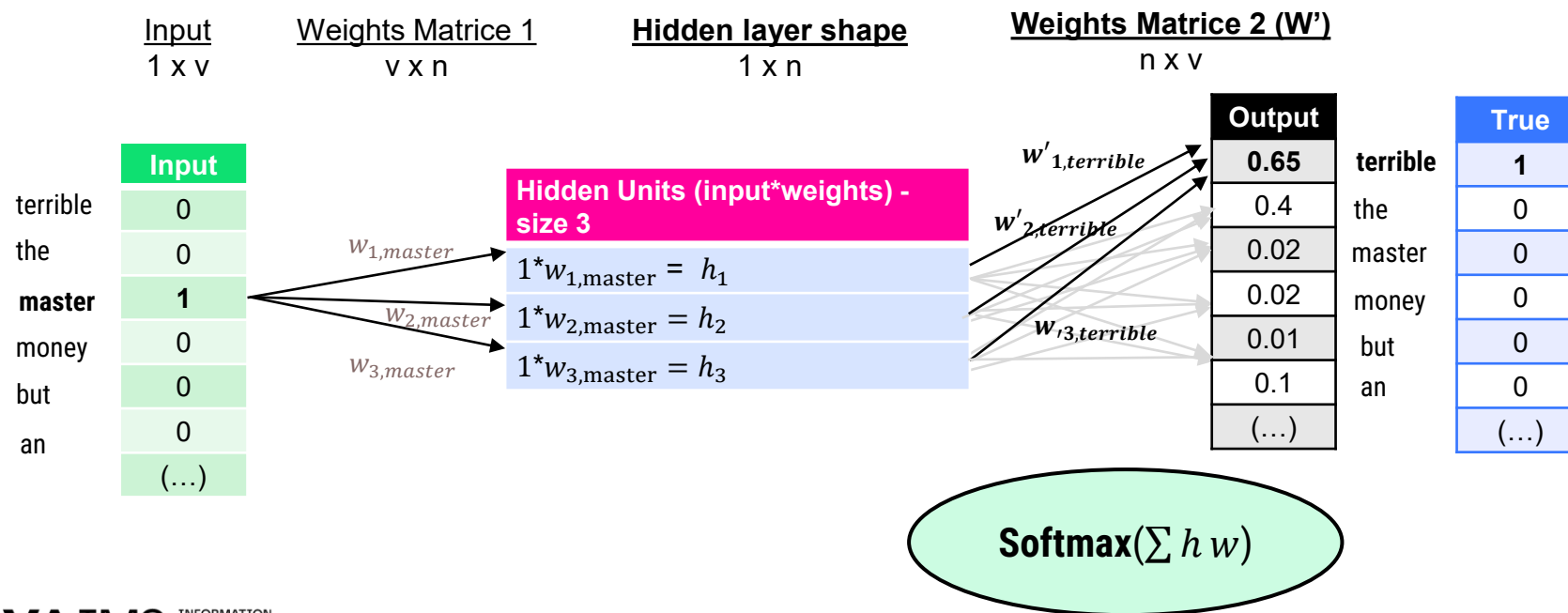
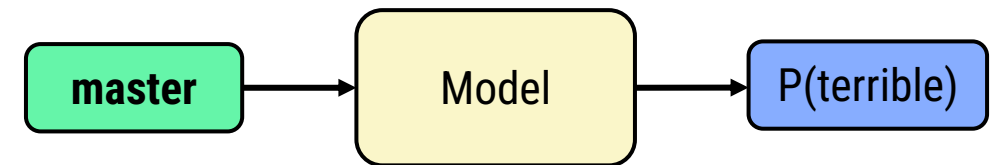
Output will be a value between 0 and 1.



4. Train the model

Multiple by a **second matrix of weights**, sum the results and transform it into **probabilities for each possible word**.

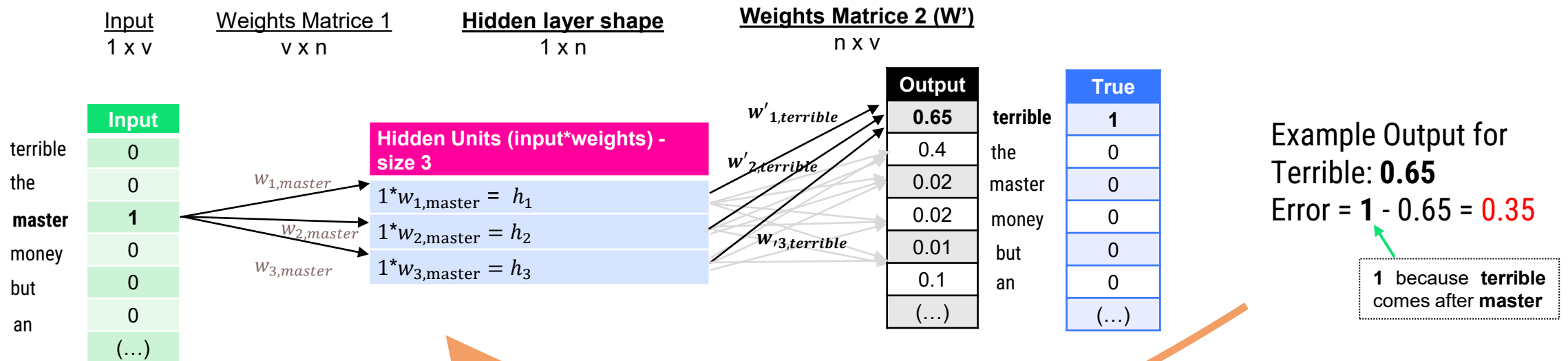
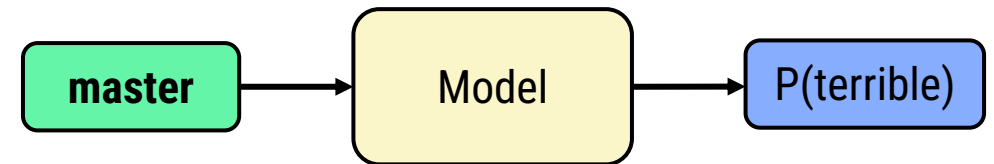
Output will be a value between 0 and 1.



4. Train the model

Multiple by a **second matrix of weights**, sum the results and transform it into **probabilities for each possible word**.

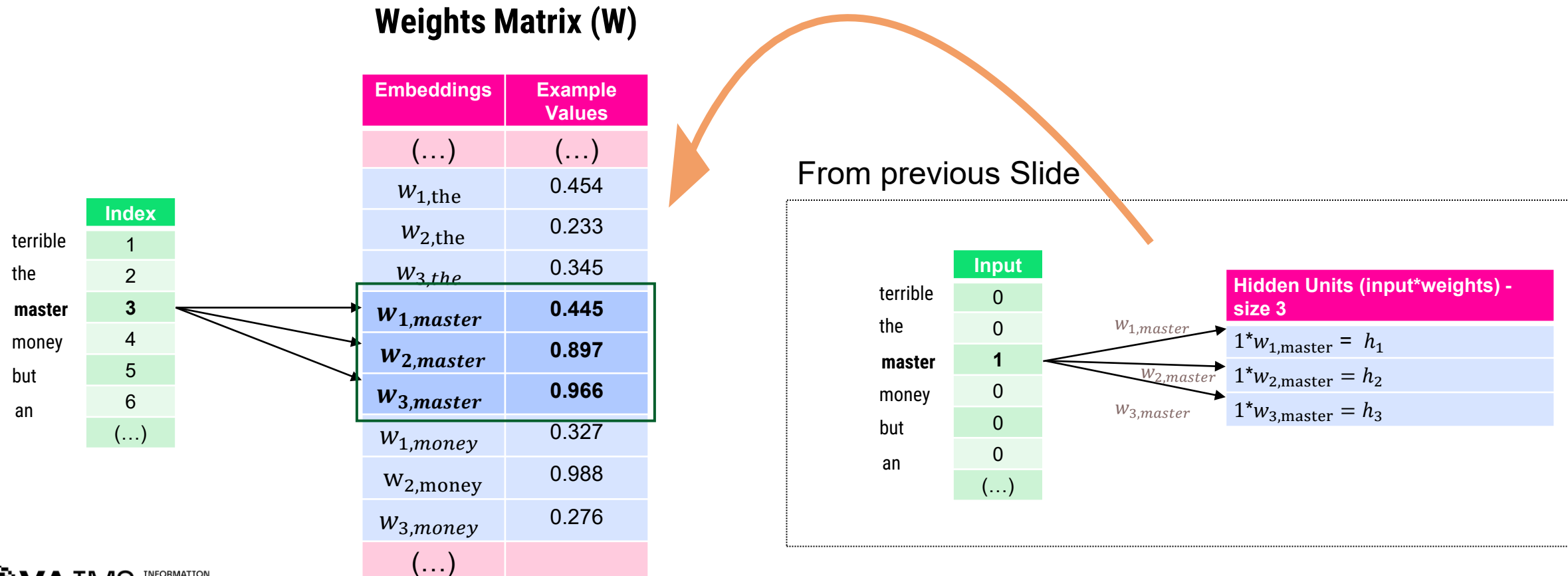
Output will be a value between 0 and 1.



5. Retrieve Embeddings

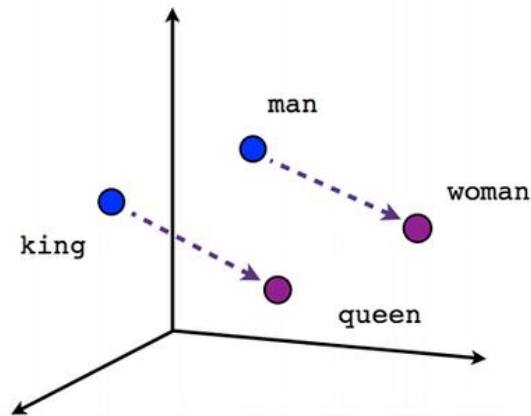
After training the model, the **embeddings** are stored in the **Weights Matrix (W)**

Master can be represented as low dimensionality vector (an embedding): **[0.445, 0.897, 0.966]**

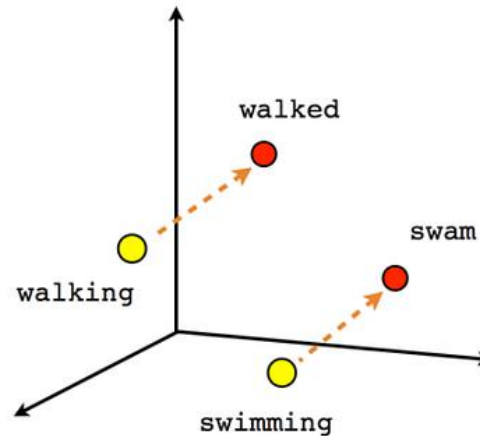


Word Embeddings Properties

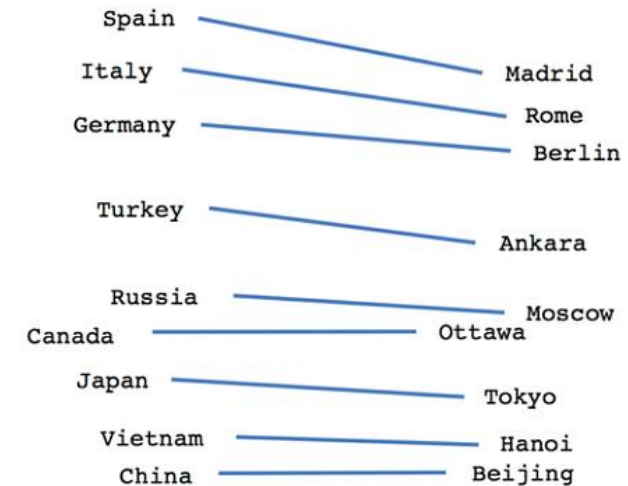
- After trained, Word Embeddings can extract **semantic relationships** between words



Male-Female



Verb tense

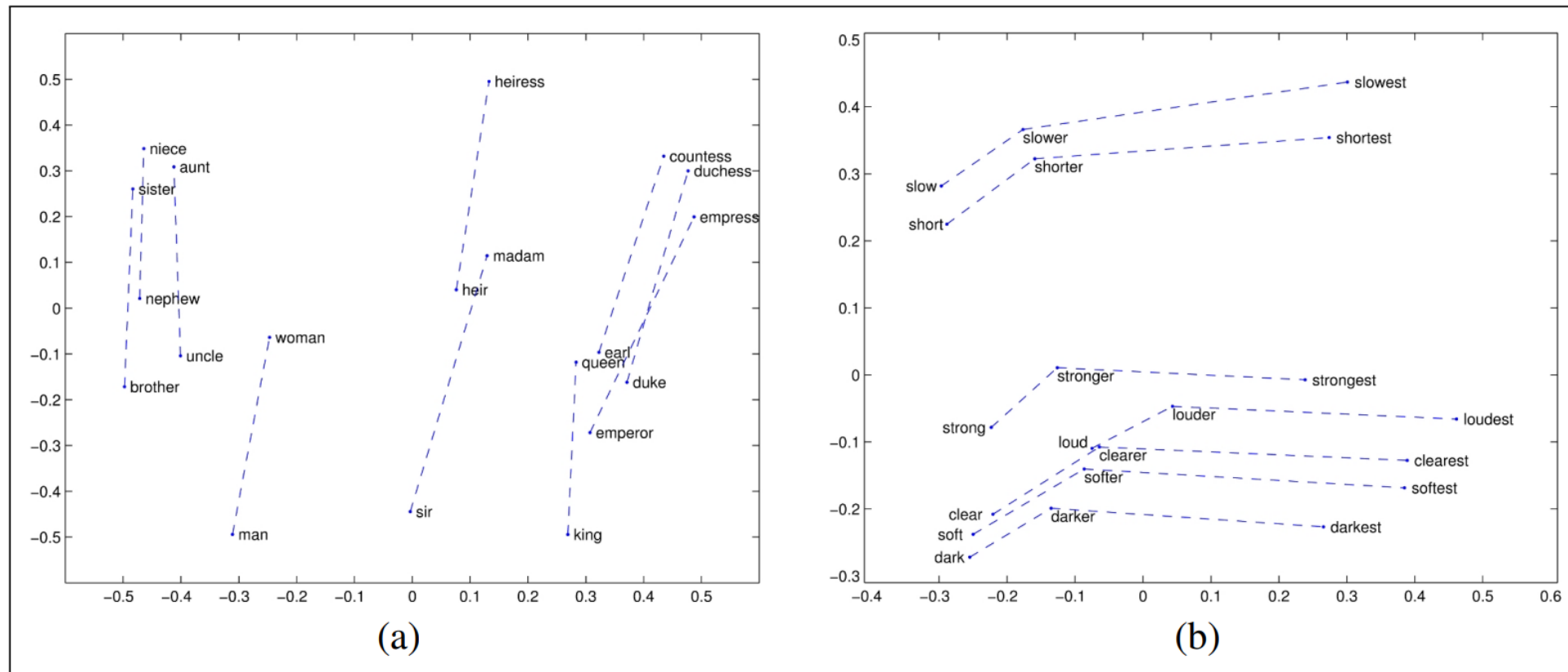


Country-Capital

<https://medium.com/data-science/creating-word-embeddings-coding-the-word2vec-algorithm-in-python-using-deep-learning-b337d0ba17a8>

Word Embeddings Properties

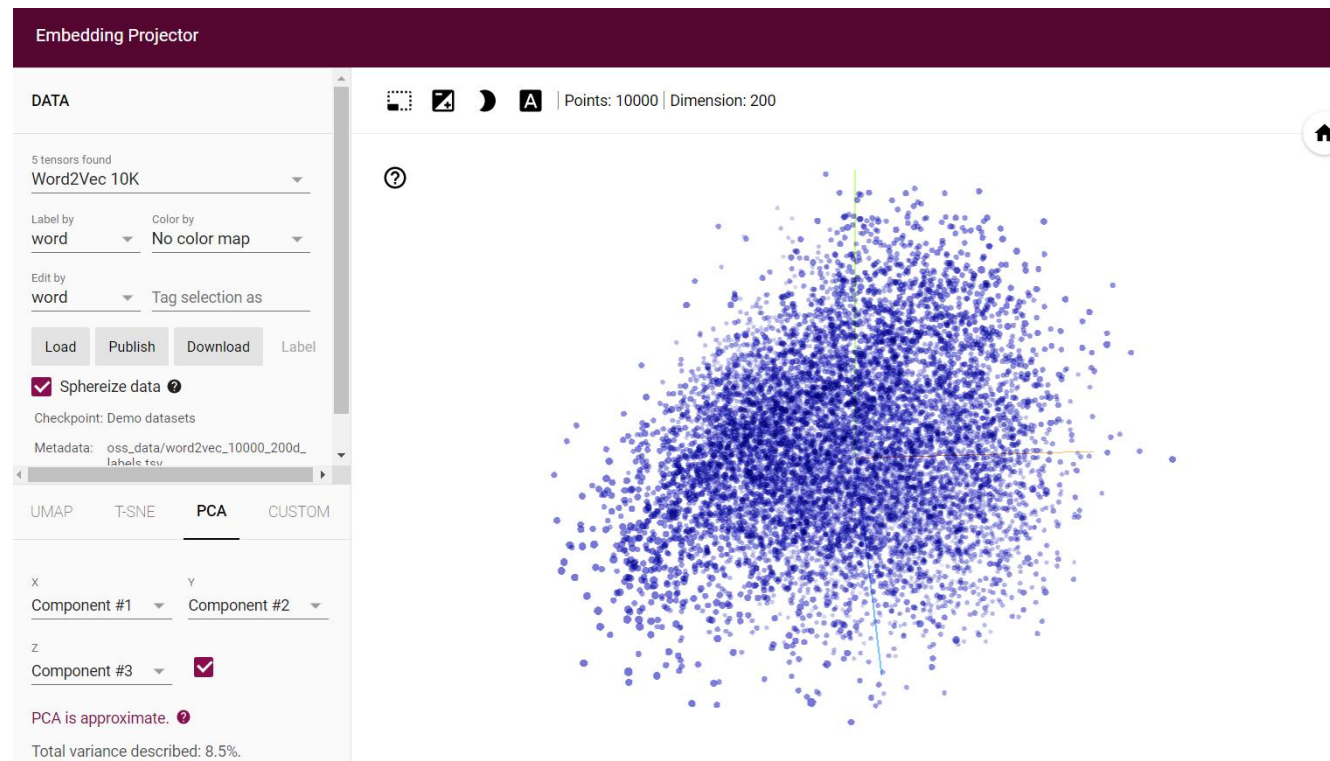
- After trained, Word Embeddings can extract **semantic relationships** between words



Word Embeddings Properties

- After trained, Word Embeddings can extract **semantic relationships** between words

<https://projector.tensorflow.org/>



Read the following for a deeper understanding of how word2vec and word embeddings work:

- Web format: <https://jalammar.github.io/illustrated-word2vec/>
- Video format: <https://www.youtube.com/watch?v=ISPIId9Lhc1g>



04. Classification with Word2Vec

Recall how to input a BoW into a Classifier

Classification with Word2Vec

Recall how to input **BoW** vectors to model:

Review 1 – “Great movie”

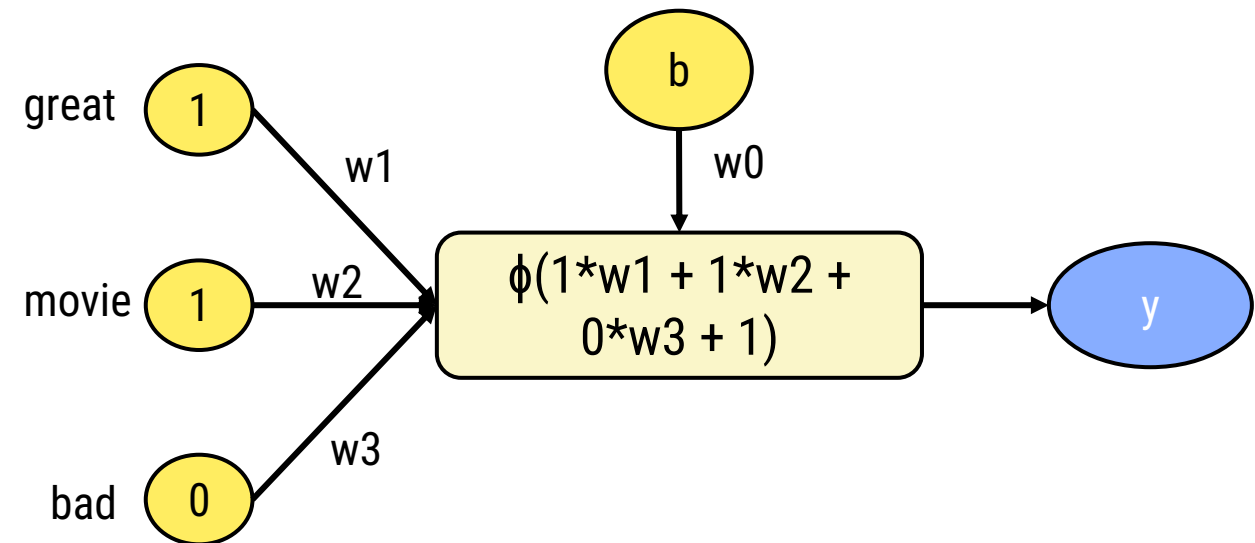
Review 2 – “Bad movie”

	great	movie	bad
R1	1	1	0
R2	0	1	1

Task:

Predict the sentiment of movie reviews using a neural network.

Review 1:



Using Word2Vec as input to a Classifier

Classification with Word2Vec

What if we consider word embeddings:

Review 1 – “Great movie”

Review 2 – “Bad movie”

	great	movie	bad
R1	[0.034, 0.235, 0.987]	[0.765, 0.523, 0.895]	-
R2	-	[0.765, 0.523, 0.895]	[0.145, 0.534, 0.556]

Note: We assume embeddings size 3 for simplicity

Task:

Predict the sentiment of movie reviews using a neural network.

Using Word2Vec as input to a Classifier

Classification with Word2Vec

How does a sentence enter the network if it's a set of vectors and not a single vector?

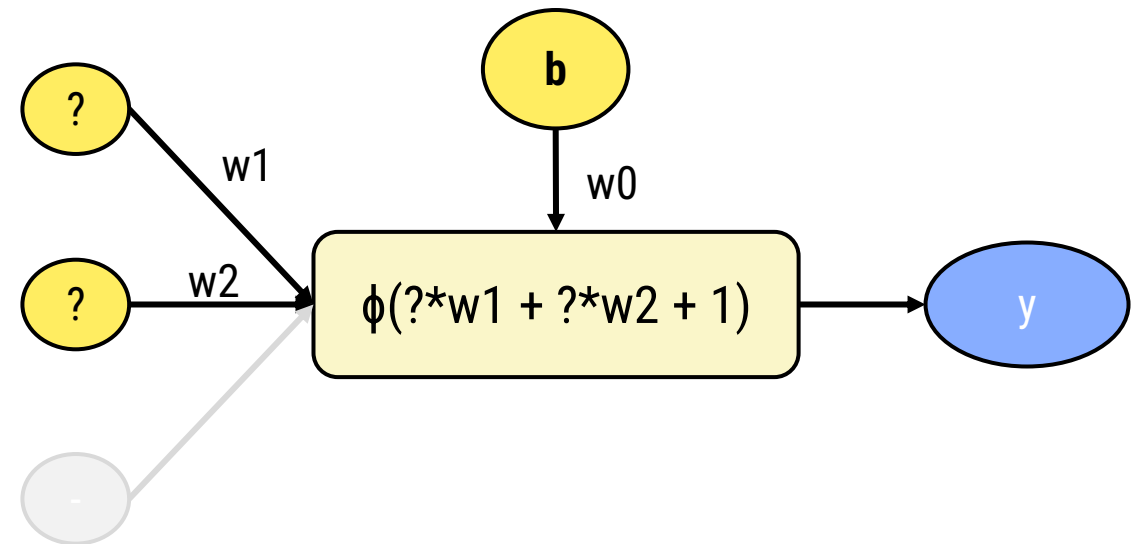
Review 1 – “Great movie”

	great	movie	bad
R1	[0.034, 0.235, 0.987]	[0.765, 0.523, 0.895]	-
R2	-	[0.765, 0.523, 0.895]	[0.145, 0.534, 0.556]

great=[0.034, 0.235, 0.987]

movie = [0.765, 0.523, 0.895]

Review 1:



One way could be to **average the vectors** of the sentence:

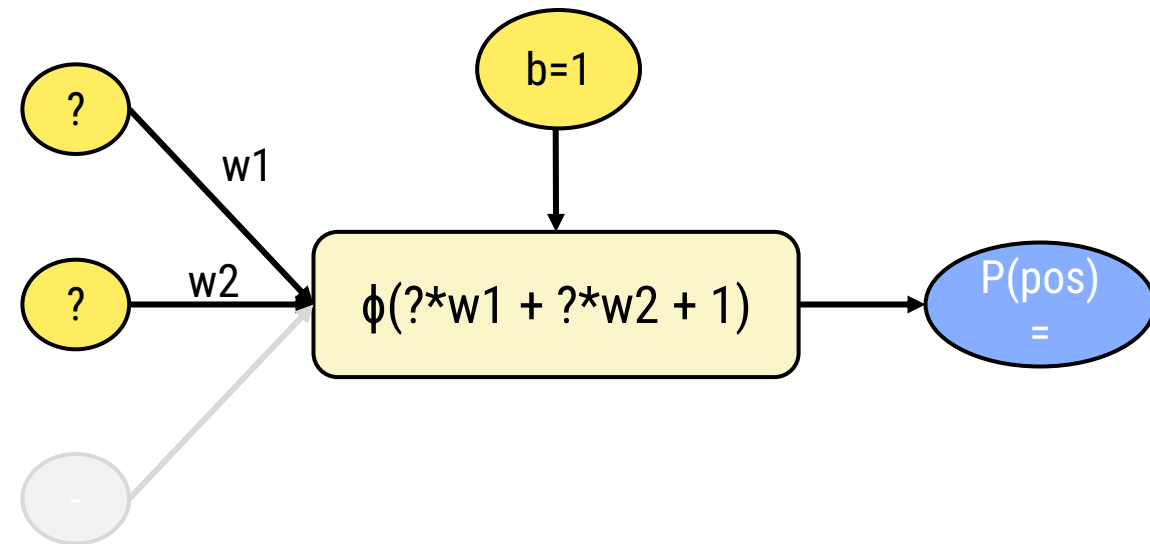
Review 1 – “Great movie”

$$(\text{great} + \text{movie}) / 2 =$$

$$= ([0.034, 0.235, 0.987] + [0.765, 0.523, 0.895]) / 2 =$$

$$= ([0.799, 0.758, 1.882]) / 2 =$$

$$= [0.399, 0.379, 0.941]$$



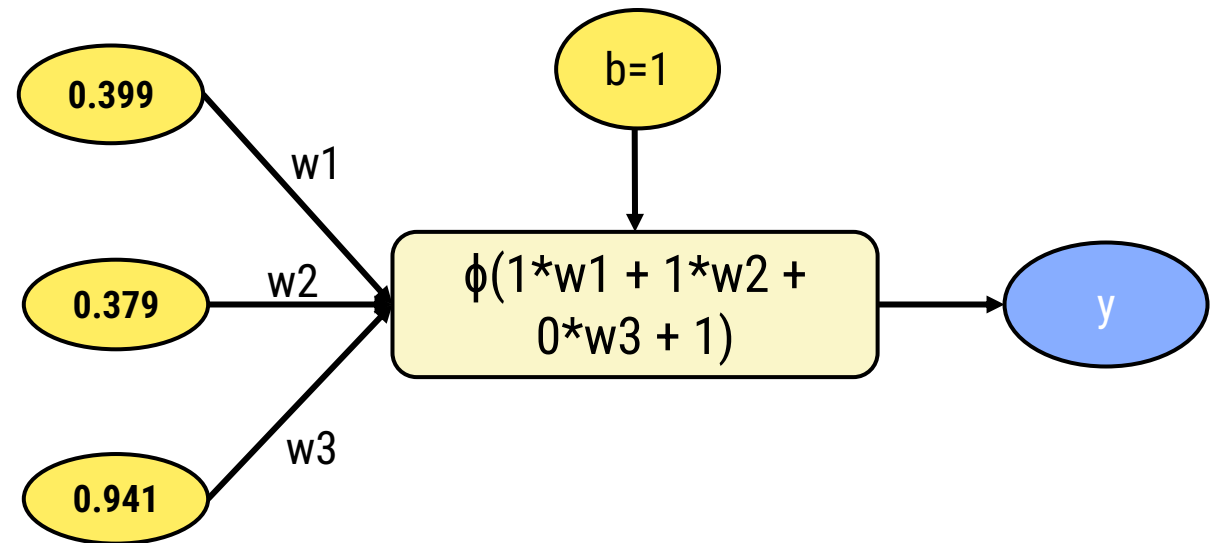
Using Word2Vec as input to a Classifier

Classification with Word2Vec

Now we have again an input able to enter the model:

Review 1 – “Great movie”

R1	0.399	0.379	0.941
----	-------	-------	-------



Using Word2Vec as input to a Classifier

Classification with Word2Vec

BoW vs Word Embeddings

Review 1 – “Great movie”

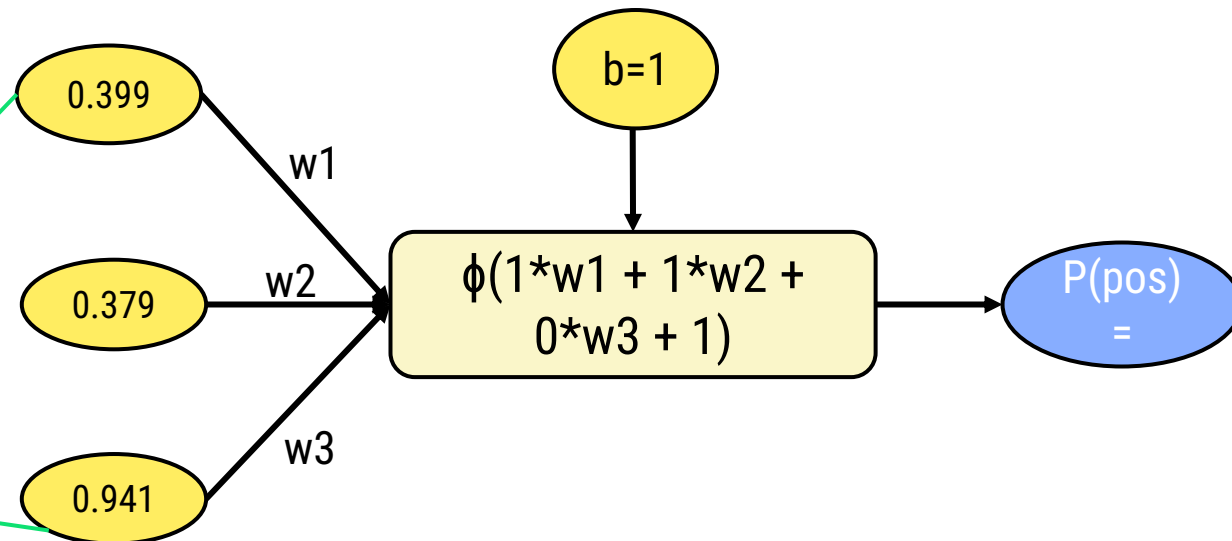
Bag-of-Words

	great	movie	bad	...	Word n
R1	1	1	0	...	0

Word Embeddings

R1	0.399	0.379	0.941
----	-------	-------	-------

Sparse vs Dense vectors!



Using Word2Vec as input to a Classifier

Classification with Word2Vec

What if we do not want to average/sum/concatenate the vectors?

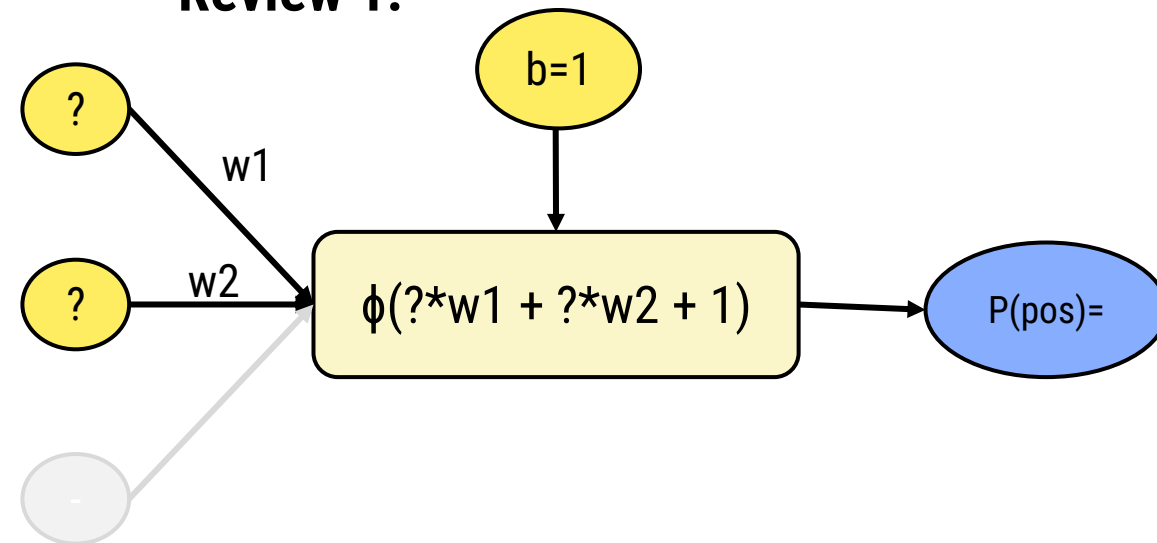
Task:

Predict the sentiment of movie reviews using a neural network.

Review 1:

great=[0.034, 0.235, 0.987]

movie = [0.765, 0.523, 0.895]

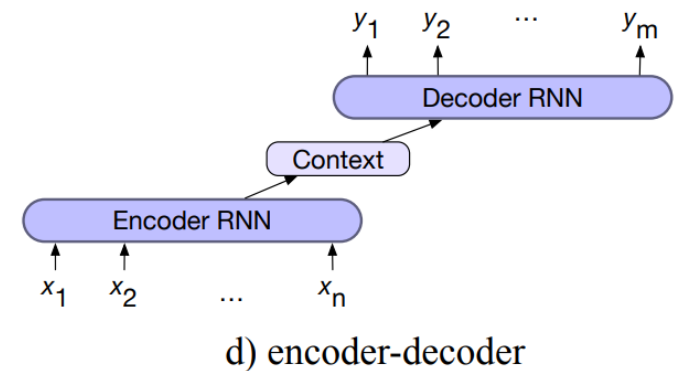
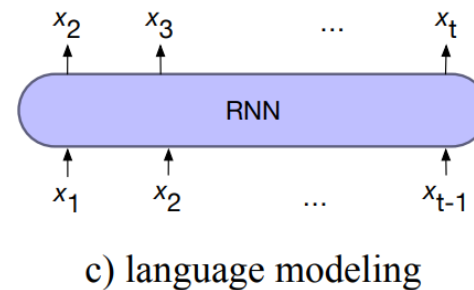
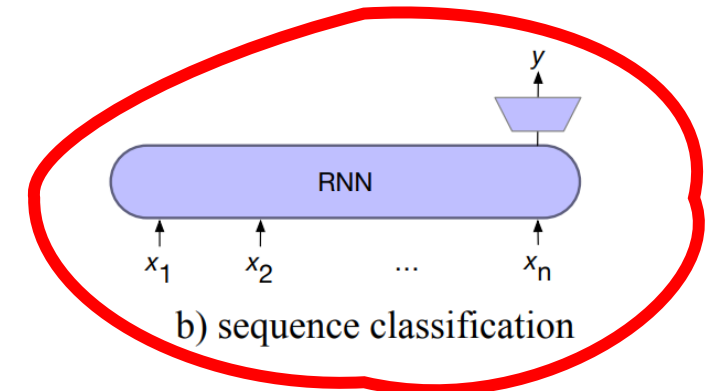
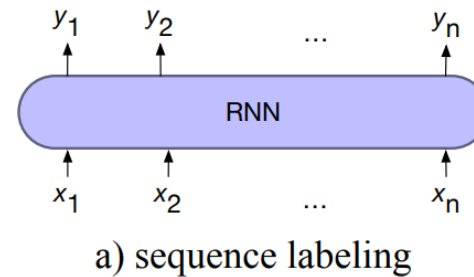


To process sequential input, we can use some well know sequential models, like **Recurrent Neural Networks (RNN)** and **Long Short Term Memory Neural Networks (LSTM)**

Common Tasks

- Next word prediction
- Sequence Labeling
- **Sequence Classification**
- Machine Translation

62





Bibliographic references:

Dan Jurafsky and James H. Martin. **Speech and Language Processing** 3rd ed.

Chapter 5

<https://web.stanford.edu/~jurafsky/slp3/>

Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). **Distributed representations of words and phrases and their compositionality**. arXiv.

<https://doi.org/10.48550/arXiv.1310.4546>

Thank you!

Acreditações e Certificações da NOVA IMS



Cofinanciado por

